

ICS2121, Métodos de Optimización 2025-2

Una breve introducción a las Heurísticas en Optimización

Qué es una Heurística

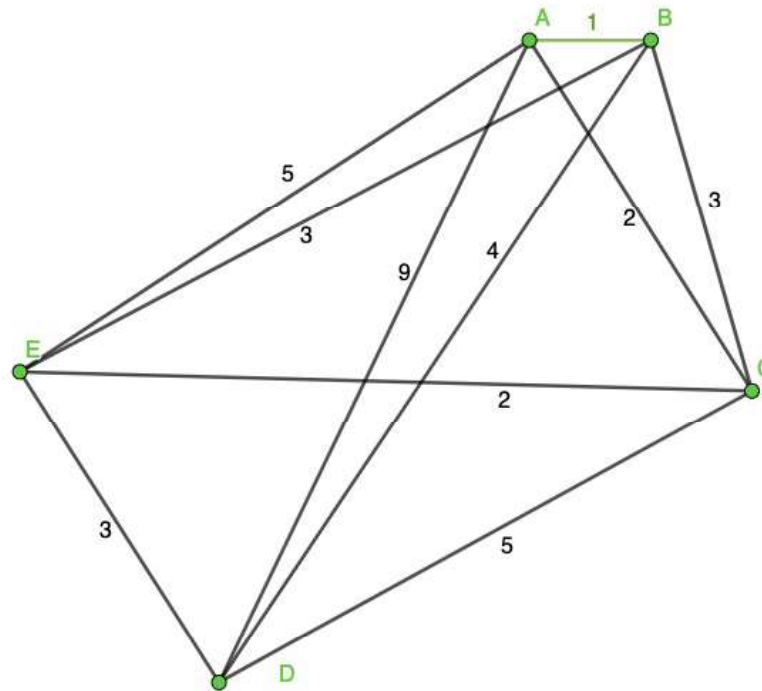
- Casi cualquier problema de optimización puede ser resuelto hasta el óptimo hoy en día.
- Pero... eso puede requerir mucho tiempo...
- Y en muchas aplicaciones, no disponemos de ese tiempo.
- Una **heurística** busca encontrar una solución factible a un problema, aunque esa solución no sea necesariamente óptima.
- Pero eso lo hacen rápido...
- Existen heurísticas específicas para muchos problemas.
- Y también hay heurísticas más generales que pueden usarse en “cualquier” problema.
- Vimos que en Relajación Lagrangiana nos gustaría usar una heurística para “factibilizar” las soluciones del problema relajado.

Ejemplo: el Problema del Vendedor Viajero

- ¿Lo recuerdan?
- El PVV es un problema difícil (NP-completo, como sabemos)
- Se puede resolver pero podríamos necesitar mucho tiempo...
- (En la próxima tarea verán un enfoque)
- Entonces, se han desarrollado varias heurísticas para abordar este problema.
- La más simple es la del “vecino más próximo”

Ejemplo: El problema del Vendedor Viajero

- Consideremos la siguiente red de ejemplo (los números sobre los arcos son los costos de viajar de un nodo a otro):

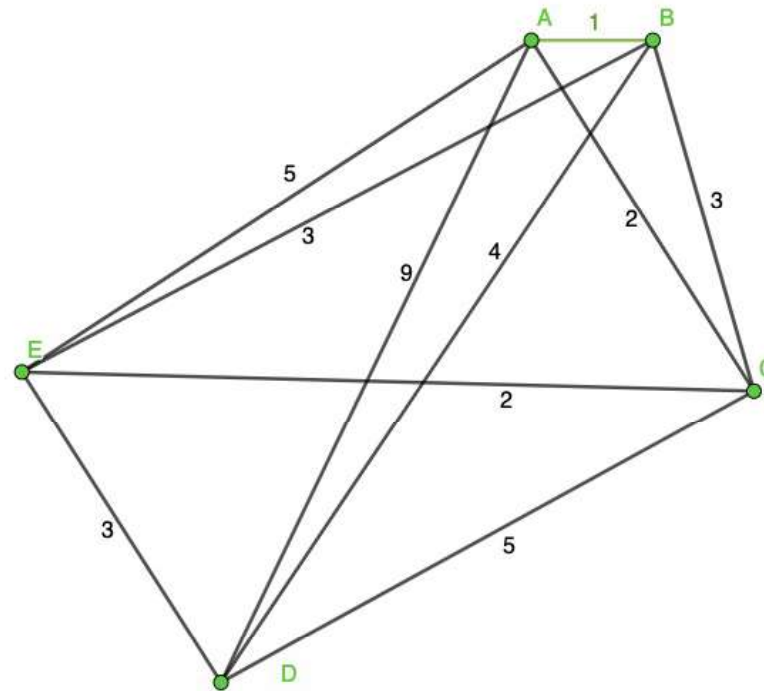


(crédito de las figuras: Denise Cariaga)

- Usemos el método del “Vecino más próximo”: ir a la ciudad aún no visitada a la que sea menos costoso viajar....

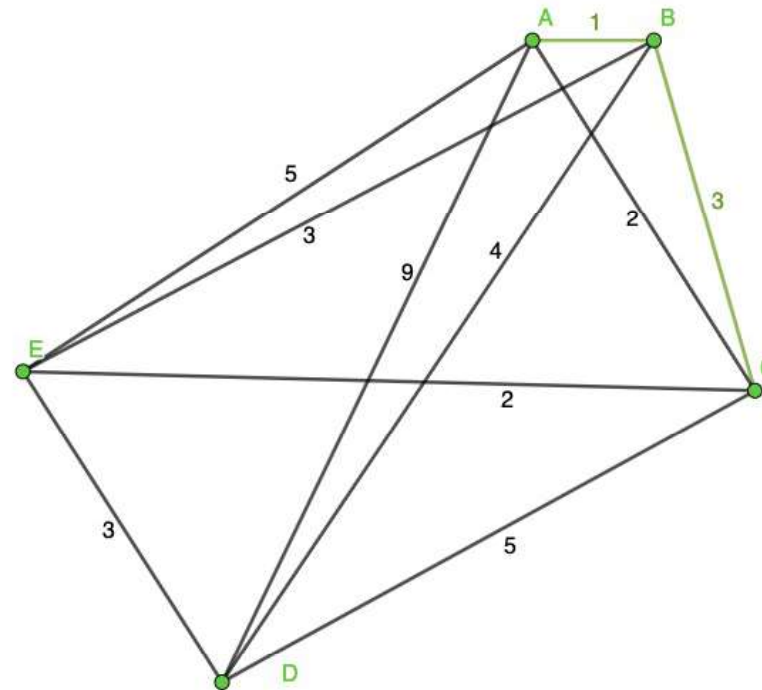
Ejemplo: El problema del Vendedor Viajero

- Esta es la secuencia de iteraciones...



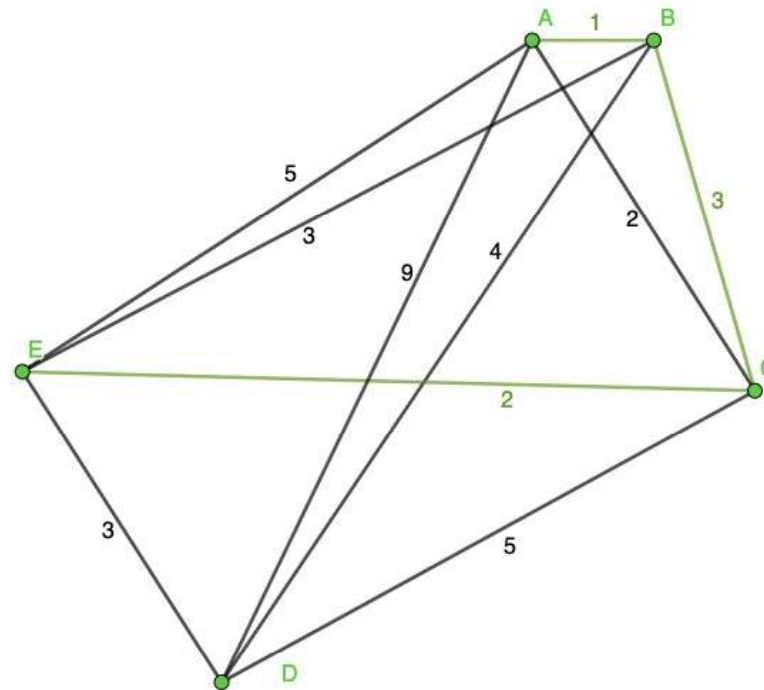
Ejemplo: El problema del Vendedor Viajero

- Esta es la secuencia de iteraciones...



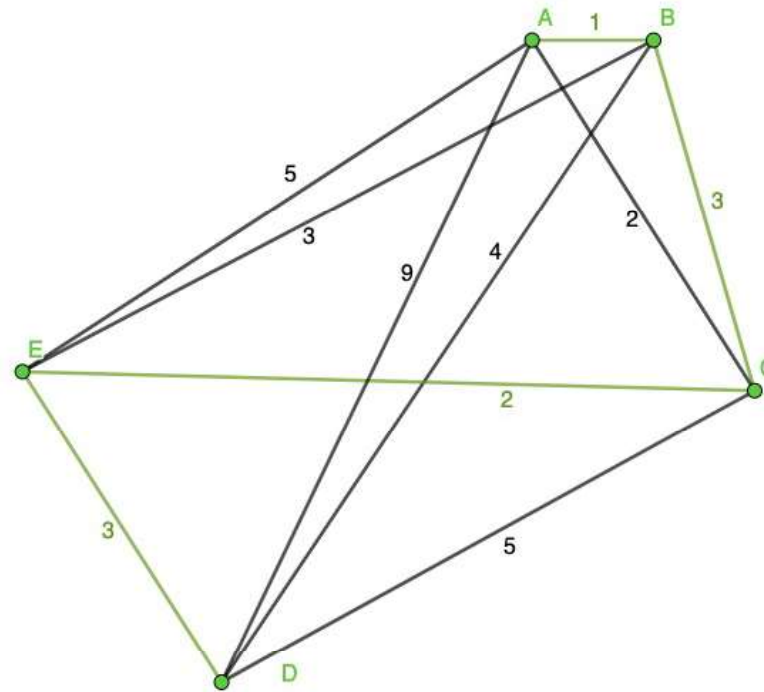
Ejemplo: El problema del Vendedor Viajero

- Esta es la secuencia de iteraciones...



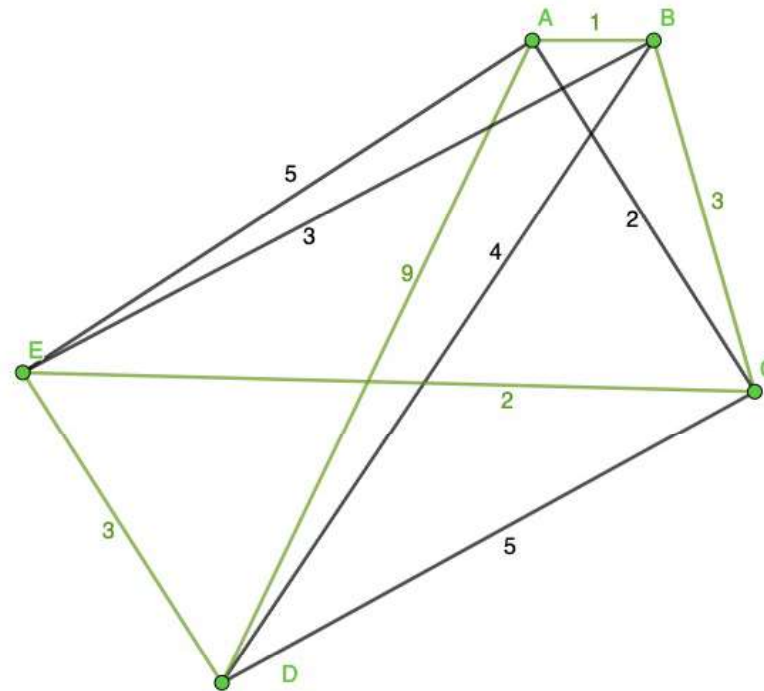
Ejemplo: El problema del Vendedor Viajero

- Esta es la secuencia de iteraciones...



Ejemplo: El problema del Vendedor Viajero

- Esta es la secuencia de iteraciones...



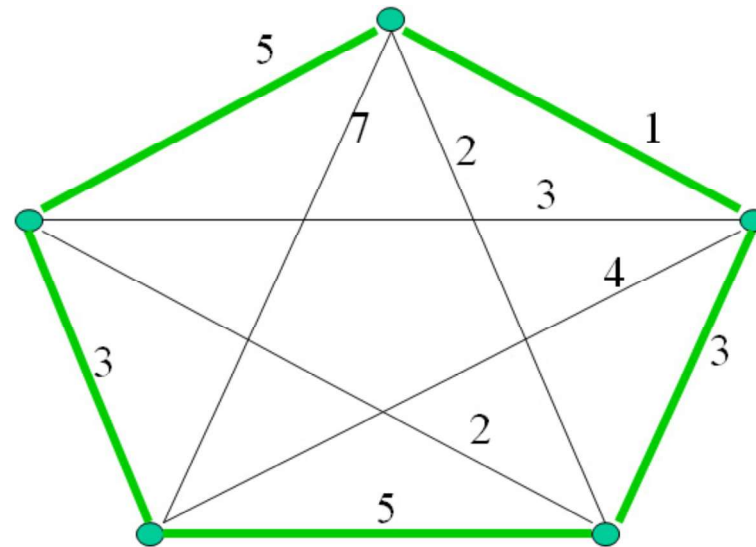
- ¿Cuánto vale el circuito? ¿hay alguna solución mejor?

Ejemplo: el Problema del Vendedor Viajero

- La anterior es un ejemplo de “**heurística golosa**” (**greedy heuristic**), que suelen ser de pobre comportamiento, pero son muy rápidas.
- Pero hay métodos más sofisticados.
- Por ejemplo, una de las más usada es 2-Opt.
- Supongamos tenemos un circuito de Vendedor Viajero válido, H (obtenido, por ejemplo con “vecino más próximo”).
- Idea: remover dos arcos no adyacentes de H y “cruzar” el circuito.
- Hacer esta evaluación para cada par de arcos no adyacentes y conservar aquel circuito generado que es el mejor en términos de costo.

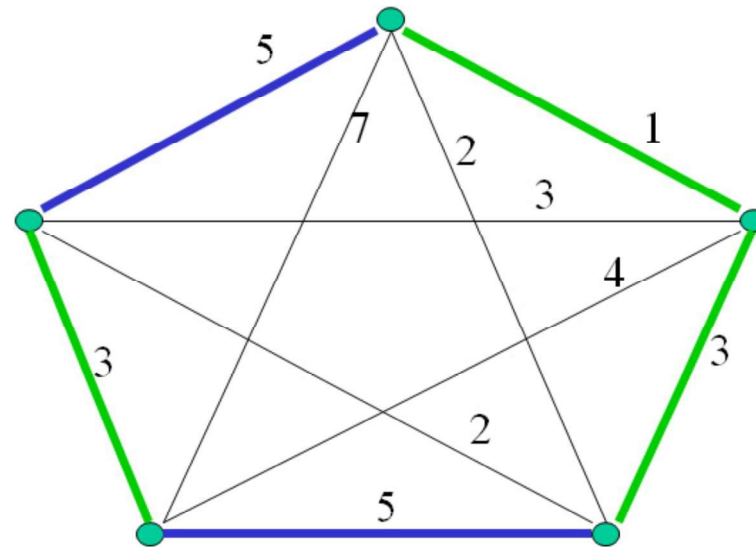
Ejemplo:

- Este ciclo no es óptimo...



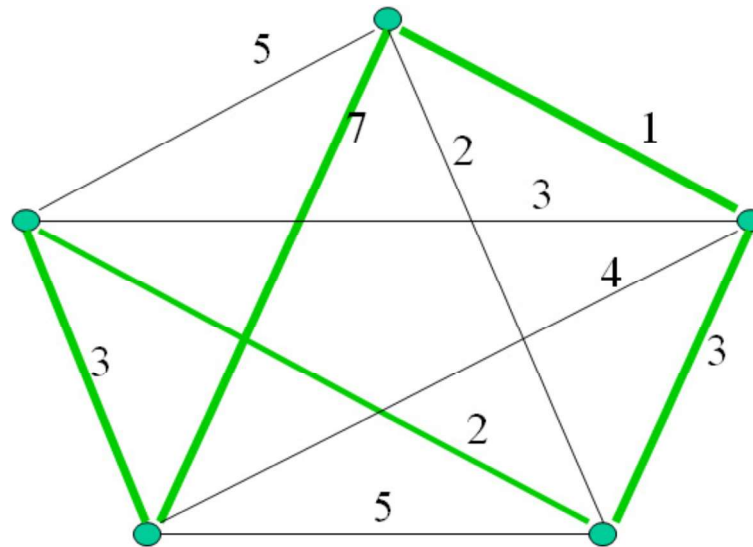
Ejemplo:

- Elegimos un par de arcos...



Ejemplo:

- “cruzamos” el ciclo...



Otras Heurísticas para el PVV

- Esto mismo se puede hacer seleccionando 3 o más arcos para cruzar pero es considerablemente más caro en tiempo.
- En ese caso, se conoce como 3-Opt, y una sofisticación de esto, combinada con 2-Opt es la famosa heurística de Lin-Kernighan.
- Variantes de esta heurística se pueden usar en el problema de Ruteo de Vehículos.
- También está la Heurística de Christofides y otras.

Hacia procedimientos generales

- Desarrollar heurísticas específicas cada vez que tenemos un problema específico es un buen camino...
- Es lo que se tiene que hacer para las heurísticas lagrangeanas, por ejemplo.
- Pero también se han desarrollado conceptos generales que permiten crear heurísticas para “casi cualquier problema”.
- Estos procedimientos son llamados “**Metaheurísticas**”:
 - ▶ Procedimientos de estructura general que se pueden aplicar a muchos problemas distintos.
 - ▶ Aunque su implementación requiere de especificaciones según el problema.

Heurísticas Generales: Metaheurísticas

- Estructura básica de todos estos procedimientos: “búsqueda local”:
- Consideremos el siguiente problema de optimización general:

$$\min_{x \in C} f(x)$$

donde C es el conjunto de soluciones factibles del problema, el cual asumimos que es discreto.

- C va a ser nuestro “espacio de búsqueda”.
- Dado $x \in C$ se define, según el problema, un conjunto $V(x) \subset C$ el cual llamamos una “vecindad” de x .
- La “vecindad” se obtiene mediante una modificación “marginal” de la solución actual.

Heurísticas Generales: Metaheurísticas

- Lo siguiente es un procedimiento de búsqueda local:

Data: Sea $x^0 \in C$, $k = 0$.

while *No se verifique el criterio de parada* **do**

 | Elegir $x^{k+1} \in V(x^k)$ tal que $f(x^{k+1}) < f(x^k)$.;

 | $k = k + 1$;

end

- El elemento clave es cómo construir la vecindad, y esto depende del problema específico.
- Ejemplo: el proceso 2-Opt para el PVV es una heurística de búsqueda local.

Heurísticas Generales: Metaheurísticas

- Problema con la búsqueda local: podemos quedar “atrapados” en un mínimo local si la etapa 1 no encuentra un punto mejor.
- Hay varias formas de resolver eso, algunas implican violar el criterio básico de selección:
 $f(x^{k+1}) < f(x^k)$.
- Explicamos ahora brevemente dos procedimientos muy usados:
 - ▶ Simulated Annealing
 - ▶ Tabu Search (Búsqueda Tabú)

Simulated Annealing

- El “Simulated Annealing” hace una exploración en el espacio de búsqueda, tratando de ir mejorando siempre la solución, pero...
- De vez en cuando considera aceptar una nueva solución x^{k+1} , incluso si $f(x^{k+1}) \geq f(x^k)$, pero eso se hace con una cierta probabilidad.
- La probabilidad es controlada de forma especial:
- Para esto vamos a definir ciertos números $T_0 > T_1 > T_2 > \dots$ tal que $T_i > 0, \forall i$ y $T_i \rightarrow 0$ con $i \rightarrow \infty$.
- Por ejemplo, una forma típica es $T_{i+1} = \alpha T_i$, con $\alpha < 1$.

Simulated Annealing

- Este es el procedimiento:
- **Simulated Annealing :**

Data: Sea $x^0 \in C$, $k = 0$.

while *No se verifique el criterio de parada* **do**

 Elegir al azar $y \in V(x^k)$.;

if $f(y) < f(x^k)$ **then**

$x^{k+1} = y$;

$k = k + 1$;

end

else

 Aceptar y con probabilidad p , donde $p = e^{-\frac{f(y)-f(x^k)}{BT_k}}$;

if y es aceptado **then**

$x^{k+1} = y$;

end

$k = k + 1$;

end

end

- Aquí, B es una constante positiva y la distribución de probabilidad original en la que se inspira esto usa $B = 1.3806503 \times 10^{-23} [JK^{-1}]$

Simulated Annealing

- La idea se inspira en el proceso físico de “annealing” en metalurgia y mecánica estadística y fue propuesto por Kirkpatrick en 1983, basado en desarrollos anteriores.
- La construcción de vecindades y selección de candidatos se suele hacer en forma aleatoria.
- La “Temperatura” y otros parámetros se calibran por simulación de MonteCarlo.
- Criterio de parada:
 - ▶ Número de iteraciones.
 - ▶ Cuando ya no hay mejora.